Adding Type Annotations to Variables

That last section was a little on the abstract side, so let's talk more concretely.

TypeScript allows you, but does not require you, to annotate your JavaScript code with information about the types of various pieces of data in the code. TypeScript uses this information as a constraint on the values that those pieces of data can contain. If you declare a variable to be a `number` and then assign it the string value `"3"`, the TypeScript compiler will complain and will not allow the code to be compiled.

If you don't add any explicit type information, however, you don't get the full benefit of using TypeScript. It will still catch some errors, such as assigning two different types of values to the same variable or if you try to send a function the wrong number of arguments. By adding as much type information as you can to your code, you will increase the value of TypeScript's type checking.

You can add type information to TypeScript in a few different ways.

When you assign a value to a variable in TypeScript using `let` or `const` (using `var` is not recommended), TypeScript assumes you intend that variable to be of the literal type of the assigned value.

If you have code that does this:

```
let index = 1
```

TypeScript infers that `index` is of type `number`. If you try to assign a different value to `index`, it will need to be a number. If you pass `index` to a function, it will only allow it if the expected type of the argument is a number.

In some cases you will know more about the type than TypeScript can infer. For example, I recently wrote the following buggy code:

```
// THIS CODE HAS A BUG
const validValues = this.existingValidValues() + "New"
```